

Báo cáo rà soát test code

<!-- framework: JUnit | run_id: run_20260617_015531_087316 -->

Framework: JUnit | Run ID: run_20260617_015531_087316

Tóm tắt kỹ thuật

- Framework: JUnit (JUnit 5)
- Kết quả chạy: 19 test, 0 thất bại, 0 lỗi, 0 bỏ qua.
- Coverage: 100 % (Jacoco) – báo cáo coverage khả dụng.
- Vấn đề chính: Một số test không phản ánh đúng yêu cầu nghiệp vụ (ví dụ: `getBalance()` trên tài khoản không hoạt động phải ném `IllegalStateException` nhưng test lại chỉ kiểm tra giá trị trả về). Điều này là blocker vì làm sai lệch tính đúng/sai của bộ kiểm thử.
- Tài liệu đã xem: `BankAccount.java` (source) và `BankAccountTest.java` (test code).

Lỗi nghiêm trọng

- Bằng chứng:
- Yêu cầu: “Calling `getBalance()` on an inactive account should trigger `IllegalStateException` via `checkActive()`.” (Requirement → validations).
- Test hiện tại:

@Test

```
void testGetBalanceOnInactiveAccount() {  
    BankAccount account = new BankAccount("ACC500", 0.0);  
    account.deactivate();  
    assertEquals(0.0, account.getBalance(), 0.001);  
}
```

(Test code trong `BankAccountTest.java`).

- Ảnh hưởng: Test không kiểm tra ngoại lệ theo yêu cầu, do đó kịch bản lỗi (TC-018) không được thực thi đúng, làm giảm độ tin cậy của bộ kiểm thử và có thể để lỗi thực tế qua.

- Cách sửa: Thay đổi test để kiểm tra `IllegalStateException` và thông điệp "Account is inactive":

@Test

```
void testGetBalanceOnInactiveAccount() {  
    BankAccount account = new BankAccount("ACC500", 0.0);  
    account.deactivate();  
    IllegalStateException ex = assertThrows(IllegalStateException.class,  
        account::getBalance);  
    assertEquals("Account is inactive", ex.getMessage());  
}
```

Test còn thiếu

- Bằng chứng:

- Yêu cầu "Calling any public method that internally uses `checkActive()` on an inactive account must throw `IllegalStateException` with message \"Account is inactive\"." (validations).
- Các test hiện có chỉ kiểm tra ngoại lệ cho `deposit`, `withdraw`, `transfer`, `deactivate` nhưng không kiểm tra thông điệp lỗi cho các phương thức này, và không có test cho `getBalance()` (đã nêu ở trên).

- Các trường hợp thiếu:

1. Kiểm tra thông điệp "Account is inactive" cho `deposit`, `withdraw`, `transfer` khi tài khoản không hoạt động.
2. Kiểm tra `IllegalStateException` cho `getBalance()` (TC-018).
3. Kiểm tra rằng `checkActive()` được gọi trong các phương thức khác (ví dụ: `withdraw`, `deposit`) bằng cách dùng `spy/mock` nếu muốn, nhưng ít nhất cần xác nhận thông điệp.

- Cách sửa: Thêm các test sau (ví dụ):

```
@Test
void testDepositInactiveMessage() {
    BankAccount acc = new BankAccount("A", 0.0);
    acc.deactivate();
    IllegalStateException ex = assertThrows(IllegalStateException.class,
    () -> acc.deposit(10));
    assertEquals("Account is inactive", ex.getMessage());
}
```

```
@Test
void testWithdrawInactiveMessage() { ... }
```

```
@Test
void testTransferInactiveSourceMessage() { ... }
```

Assertion yếu

- Bằng chứng:
- Các test `testDepositIntoInactiveAccount`, `testWithdrawFromInactiveAccount`, `testTransferToInactiveTarget` chỉ dùng `assertThrows` mà không kiểm tra nội dung thông điệp ngoại lệ.
- Test `testGetBalanceOnInactiveAccount` (đã nêu) không kiểm tra ngoại lệ mà chỉ so sánh giá trị trả về.

- Ảnh hưởng: Không phát hiện được trường hợp thông điệp sai (có thể thay đổi mà vẫn pass).

- Cách sửa: Thêm `assertEquals("Account is inactive", ex.getMessage())` cho mọi `assertThrows` liên quan tới `IllegalStateException` do `checkActive()`.

Rủi ro maintainability

- Bằng chứng:
- Các test tạo tài khoản mới trong mỗi phương thức, không dùng `@BeforeEach` để tái sử

dụng đối tượng, dẫn tới lặp lại mã và dễ gây lỗi khi thay đổi logic khởi tạo.

- Không có comment hoặc naming rõ ràng cho các biến test (ví dụ: account, source, target là ổn, nhưng một số test không mô tả rõ mục đích).
- Ảnh hưởng: Khi lớp BankAccount mở rộng (thêm trường, thay đổi constructor), sẽ phải cập nhật nhiều test thủ công.
- Cách sửa: Sử dụng @BeforeEach để khởi tạo các đối tượng chuẩn, và đặt tên test mô tả hơn (đã khá, nhưng có thể cải thiện). Ví dụ:

```
private BankAccount activeAccount;  
private BankAccount inactiveAccount;
```

```
@BeforeEach  
void setUp() {  
    activeAccount = new BankAccount("ACT", 100.0);  
    inactiveAccount = new BankAccount("INA", 0.0);  
    inactiveAccount.deactivate();  
}
```

Gợi ý sửa ngay

1. Sửa test testGetBalanceOnInactiveAccount để kiểm tra ngoại lệ và thông điệp (như trong phần *Lỗi nghiêm trọng*).
2. Bổ sung kiểm tra thông điệp cho tất cả các assertThrows liên quan tới IllegalStateException (deposit, withdraw, transfer).
3. Thêm test cho TC-018 (đã nêu) và các test xác nhận thông điệp "Account is inactive" cho các phương thức công cộng.
4. Tối ưu hoá setup bằng @BeforeEach để giảm lặp lại mã khởi tạo tài khoản.
5. Cập nhật tên và comment nếu cần để tăng tính đọc hiểu.

Nếu thực hiện các sửa đổi trên, bộ kiểm thử sẽ phản ánh đầy đủ yêu cầu nghiệp vụ, tăng độ bao phủ thực tế và giảm rủi ro lỗi trong tương lai.